

ROS 2 Jazzy Perception Module Guide

Starter-Friendly Developer Reference & Categorized Testing Manual for Terrain Geometry, ArUco 3D Vision & System Launching

Target ROS 2 Distro: ROS 2 Jazzy Jalisco	Hardware Target: NVIDIA Jetson Orin Nano (8GB) / PC
Camera Sensor: Intel RealSense D435 RGB-D	Simulation World: world_Rotated_Aruco.world
Primary Workspace Path: /home/saif/Desktop/MESEKET/Autonmous-27/Autonmous_Ws	Primary Package: terrain_geometry

1. Starter-Friendly ROS 2 Perception Concepts

If you are new to ROS 2 or robotics vision, here is a quick overview of the key concepts used throughout this guide:

- **ROS 2 Node:** An executable process running a specific task (e.g. terrain_node filters point clouds, aruco_detector_node detects markers).
- **ROS 2 Topic:** A named data highway where nodes stream messages (e.g. /camera/depth/color/points transfers 3D point cloud data).
- **TF Tree (Transform Tree):** ROS keep tracks of 3D spatial relationships between coordinate frames. For example, base_link → camera_link → camera_depth_optical_frame tells ROS where the camera is located relative to the physical center of the rover.
- **QoS (Quality of Service):** Rules governing how messages are delivered. **Best Effort** drops old frames to prioritize speed (used for camera video/point clouds). **Reliable** guarantees delivery (used for costmaps and obstacle markers).

2. Perception Module Edits for SLAM & Nav2 Alignment

To ensure the Perception module interfaces cleanly with **Saif SLAM** (RTAB-Map + EKF) and **Nav2 Path Planning**, the following exact topics, frame IDs, and driver settings must be enforced:

Interface Boundary	Topic Name	Message Type	QoS Setting	Alignment & Purpose
ArUco → SLAM Reset	/perception/aruco_pose	geometry_msgs/msg/PoseStamped	Reliable	Provides 3D target marker pose to RTAB-Map to trigger global landmark drift resets.
Obstacles → Nav2 Costmap	/perception/obstacles_only	vision_msgs/msg/Detection3DArray	Reliable	Streams persistent 3D rock bounding boxes to Nav2's costmap_2d server for MPPI steering.
Local Terrain Costmap	/terrain/costmap	nav_msgs/msg/OccupancyGrid	Reliable	Outputs 2D grid pre-inflated by robot footprint radius (0.3m) plus safety buffer.
Camera Driver Setting	/camera/depth/color/points	sensor_msgs/msg/PointCloud2	Best Effort	RealSense driver MUST launch with enable_pointcloud:=true to publish 3D points.

3. Git Configuration, Repository Setup & Branch Workflow

3.1. Setting Up Git in a Fresh Directory

Configure your identity and initial repository setup in a new folder:

```
git config --global user.name "Your Name"
git config --global user.email "your.email@example.com"
cd /home/saif/Desktop/MESEKET/Autonmous-27
git init
git remote add origin git@github.com:your-org/Autonmous-27.git
```

3.2. Pulling the Current Perception Code Branch

Fetch all remote branches and checkout the feature branch containing the active perception code:

```
cd /home/saif/Desktop/MESEKET/Autonmous-27/Autonmous_Ws
git fetch origin
git checkout main # or git checkout perception/task-3a-aruco-node
git pull origin main
```

3.3. Committing and Pushing Changes Back to GitHub

After editing files or adding new node scripts, push changes back to the same branch from the terminal:

```
# Check status of modified files
git status

# Stage perception package changes
git add Perception/terrain_geometry_improved/

# Commit changes with clean message
git commit -m "feat(perception): integrate aruco detector with terrain geometry system launch"

# Push changes back to origin branch
git push origin main # or git push origin <your-branch-name>
```

4. ROS 2 Jazzy Build & Sourcing Commands

Always build from the main repository workspace root after sourcing ROS 2 Jazzy base setup:

```
# Always execute from workspace root
cd /home/saif/Desktop/MESEKET/Autonmous-27

# Source ROS 2 Jazzy system setup
source /opt/ros/jazzy/setup.bash

# Build perception interface and algorithm packages
colcon build --symlink-install --packages-select terrain_geometry_msgs terrain_geometry

# Source workspace overlay setup
source install/setup.bash
```

5. Categorized Standalone Testing Manual

To easily debug and validate the perception module, follow these categorized standalone testing procedures:

Category A: Testing Terrain Geometry (Standalone Verification)

Objective: Validate Patchwork++/RANSAC ground removal, Voxel downsampling, DBSCAN rock clustering, and local costmap rasterization.

```
# Step A.1: Launch Gazebo World (Terminal 1)
cd /home/saif/Desktop/MESEKET/Autonmous-27
```

```
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch worlds world1.launch.py

# Step A.2: Spawn Rover (Terminal 2)
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch my_robot_description spawn_rover.launch.py

# Step A.3: Launch Terrain Geometry Standalone (Terminal 3)
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch terrain_geometry terrain.launch.py

# Step A.4: Launch Teleop (Terminal 4)
ros2 run teleop_twist_keyboard teleop_twist_keyboard

# Step A.5: Echo Costmap & Feature Topics (Terminal 5)
ros2 topic echo /terrain/costmap nav_msgs/msg/OccupancyGrid
ros2 topic echo /terrain/obstacle_features terrain_geometry_msgs/msg/ObstacleFeatureArray
```

Expect to See (Terrain Verification): /terrain/costmap outputs occupancy grids with 0.05m cells. /terrain/obstacle_features outputs bounding box centroids and sizes matching physical Gazebo rocks.

Category B: Testing ArUco Marker Detection (Standalone Verification)

Objective: Validate 2D corner extraction, OpenCV SolvePnP 3D pose math, and target pose output.

```
# Step B.1: Launch Gazebo World with ArUco Markers (Terminal 1)
cd /home/saif/Desktop/MESEKET/Autonomous-27
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch worlds world_Rotated_Aruco.launch.py

# Step B.2: Spawn Rover (Terminal 2)
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch my_robot_description spawn_rover.launch.py

# Step B.3: Run ArUco Detector Node Standalone (Terminal 3)
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 run terrain_geometry aruco_detector_node

# Step B.4: Echo Target 3D Pose (Terminal 4)
ros2 topic echo /perception/aruco_pose geometry_msgs/msg/PoseStamped
ros2 topic hz /perception/aruco_pose
```

Expect to See (ArUco Verification): Steady ~10-15 Hz output on /perception/aruco_pose detailing 3D translation (\$X, Y, Z\$) and orientation. 3D distance error MUST be $\leq 2 \text{ cm}$ at 2.0 m distance in simulation.

Category C: Testing System Integration & Memory Retention

Objective: Validate multi-node launcher (perception_system.launch.py), Persistent Memory Node EMA tracking, and QoS compatibility.

```
# Step C.1: Launch Gazebo World & Spawn Rover (Terminal 1)
ros2 launch my_robot_description gazebo.launch.py world:=world_Rotated_Aruco.world

# Step C.2: Launch Full Perception System (Terminal 2)
ros2 launch terrain_geometry perception_system.launch.py

# Step C.3: Verify Active Nodes (Terminal 3)
ros2 node list
# EXPECTED: /terrain_node, /persistent_memory_node, /aruco_detector_node

# Step C.4: Memory Retention Blind Spot Test
```

```
# Drive rover near a rock, then rotate camera 90 degrees away into a blind spot.
ros2 topic echo /perception/obstacles_only vision_msgs/msg/Detection3DArray
```

Expect to See (Integration Verification): Rocks remain published on /perception/obstacles_only for 5 seconds after leaving camera view. ros2 doctor --report shows zero QoS mismatch warnings.

6. RViz2 Display Master Reference Table

Run rviz2 from terminal. Set **Fixed Frame** to base_link or odom, then configure the displays below:

Display Name	RViz Display Type	Topic	QoS Reliability	What You Will See in RViz
Raw RGB Feed	Image	/camera/color/image_raw	Best Effort	Live 2D video feed from front camera showing ArUco marker.
Raw PointCloud	PointCloud2	/camera/depth/color/points	Best Effort	Colored 3D point cloud of terrain in front of rover.
ArUco Target Pose	Pose / TF	/perception/aruco_pose	Reliable	3D orientation arrow and coordinate axes anchored on marker.
Local Costmap	OccupancyGrid	/terrain/costmap	Reliable	2D grid showing clear space (white) and inflated cost zones (red).
3D Obstacle Boxes	MarkerArray	/terrain/obstacle_markers	Reliable	3D bounding boxes around rocks labeled with Cluster ID & distance.
Debug Ground	PointCloud2	/terrain/debug/ground_cloud	Best Effort	Visualizes ground points separated by Patchwork++ (solid green).
Debug Clusters	PointCloud2	/terrain/debug/clustered_cloud	Best Effort	Visualizes obstacle points colored by DBSCAN Cluster ID.

7. Unified Launch Script & Single-Script Execution

Create perception_system.launch.py inside terrain_geometry/launch/:

```
# launch/perception_system.launch.py
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node

def generate_launch_description():
    pkg_dir = get_package_share_directory('terrain_geometry')
    param_file = os.path.join(pkg_dir, 'config', 'perception_params.yaml')

    return LaunchDescription([
        Node(package='terrain_geometry', executable='terrain_node',
            name='terrain_node', parameters=[param_file]),
        Node(package='terrain_geometry', executable='persistent_memory_node',
            name='persistent_memory_node', parameters=[param_file]),
        Node(package='terrain_geometry', executable='aruco_detector_node',
            name='aruco_detector_node', parameters=[param_file]),
    ])

```

Firing the Whole Module with One Command:

```
# Single Command to Fire Entire Perception System
cd /home/saif/Desktop/MESEKET/Autonomous-27
source /opt/ros/jazzy/setup.bash && source install/setup.bash
ros2 launch terrain_geometry perception_system.launch.py
```

8. Starter Troubleshooting & FAQs

- **Q: RViz shows a status warning 'No image/pointcloud received'!**

Fix: Check the display settings dropdown in RViz. Change **Reliability Policy** from *Reliable* to **Best Effort**. RealSense drivers publish image/pointcloud topics with Best Effort QoS.

- **Q: colcon build throws package not found errors!**

Fix: Make sure you sourced `/opt/ros/jazzy/setup.bash` in your terminal before building, and build from workspace root `/home/saif/Desktop/MESEKET/Autonmous-27`.

- **Q: `/perception/aruco_pose` is empty!**

Fix: Check if the rover is close enough to the ArUco marker (within 4.0m) and the marker is visible in `/camera/color/image_raw`.